To complete **Stage 3: update and delete with SQL**, you need to update your remaining `PUT` and `DELETE` endpoints. These routes will now run parameterized `UPDATE` and `DELETE FROM` SQL strings directly against your database, rather than altering memory arrays.

Choose the code block below that matches your project lane.

Node.js Lane (Express + better-sqlite3)

Update the modification routes inside your `server.js` file. You can check the `.changes` property on the database operation object to see if any rows were matched and updated or deleted. If `.changes === 0`, it means the ID does not exist, and you should return a `404` error.

javascript

```
// ========================================
// Stage 3: PUT /tasks/:id (Update Task)
// ========================================
app.get('/tasks/:id', (req, res) => { /* Code from Stage 1 */ });

app.put('/tasks/:id', (req, res) => {
  const taskId = parseInt(req.params.id);
  const { title, done } = req.body;

  // Validation: Missing all fields
  if (title === undefined && done === undefined) {
    return res.status(400).json({ error: "Missing fields to update" });
  }

  // Validation: Empty title text provided
  if (title !== undefined && (!title || title.trim() === "")) {
    return res.status(400).json({ error: "Title cannot be empty" });
  }

  // Validation: Invalid done parameter type
  if (done !== undefined && typeof done !== "boolean") {
    return res.status(400).json({ error: "Done must be a boolean value" });
  }

  // First fetch the existing task record to pull baseline values
  const currentTask = db.prepare('SELECT * FROM tasks WHERE id = ?').get(taskId);
  if (!currentTask) {
```

```
    return res.status(404).json({ error: `Task ${taskId} not found` });
  }

  // Determine fallback field settings
  const finalTitle = title !== undefined ? title : currentTask.title;
  const finalDone = done !== undefined ? (done ? 1 : 0) : currentTask.done;

  // Run the safe SQL update query string
  db.prepare('UPDATE tasks SET title = ?, done = ? WHERE id = ?').run(finalTitle, finalDon

  res.json({
    id: taskId,
    title: finalTitle,
    done: Boolean(finalDone)
  });
});

// ========================================
// Stage 3: DELETE /tasks/:id (Delete Task)
// ========================================
app.delete('/tasks/:id', (req, res) => {
  const taskId = parseInt(req.params.id);

  const result = db.prepare('DELETE FROM tasks WHERE id = ?').run(taskId);

  // If no database rows changed, the ID did not exist
  if (result.changes === 0) {
    return res.status(404).json({ error: `Task ${taskId} not found` });
  }

  // Return 204 No Content for a successful deletion
  res.status(204).send();
});
```

Use code with caution.

🟡 Python Lane (FastAPI + sqlite3)

Update your matching endpoints inside your `main.py` file. Use the `.rowcount` property on your database cursor object to check if a row was found and modified or deleted. Remember to call `conn.commit()` to save your database changes permanently.

python

```python
# ========================================
# Stage 3: PUT /tasks/{id} (Update Task)
# ========================================
@app.put("/tasks/{id}")
def update_task(id: int, task_input: TaskUpdate):
    # Validation: Missing all update payload properties
    if task_input.title is None and task_input.done is None:
        raise HTTPException(status_code=400, detail={"error": "Missing fields"})

    # Validation: Empty title text provided
    if task_input.title is not None and not task_input.title.strip():
        raise HTTPException(status_code=400, detail={"error": "Empty title"})

    conn = get_db_connection()
    cursor = conn.cursor()

    # Step 1: Read current task state to handle patch updates safely
    cursor.execute("SELECT * FROM tasks WHERE id = ?", (id,))
    row = cursor.fetchone()
    if row is None:
        conn.close()
        raise HTTPException(status_code=404, detail={"error": f"Task {id} not found"})

    # Determine fallback values
    final_title = task_input.title if task_input.title is not None else row["title"]
    final_done = 1 if (task_input.done if task_input.done is not None else bool(row["done"

    # Step 2: Execute permanent database update
    cursor.execute(
        "UPDATE tasks SET title = ?, done = ? WHERE id = ?",
        (final_title, final_done, id)
    )
    conn.commit()
    conn.close()

    return {
        "id": id,
```

```
        "title": final_title,
        "done": bool(final_done)
    }

# ========================================
# Stage 3: DELETE /tasks/{id} (Delete Task)
# ========================================
@app.delete("/tasks/{id}", status_code=status.HTTP_204_NO_CONTENT)
def delete_task(id: int):
    conn = get_db_connection()
    cursor = conn.cursor()

    cursor.execute("DELETE FROM tasks WHERE id = ?", (id,))
    conn.commit()

    # Check if a row was actually deleted
    if cursor.rowcount == 0:
        conn.close()
        raise HTTPException(status_code=404, detail={"error": f"Task {id} not found"})

    conn.close()
    return Response(status_code=status.HTTP_204_NO_CONTENT)
```

Use code with caution.

🏁 Checkpoint Verification

Run these validation tests in your terminal to verify that your CRUD API works correctly with the database:

1. **Create an initial task:**

```
bash
curl -i -X POST -H "Content-Type: application/json" -d '{"title":"Database Testing"}'
# Note down the returned ID (for example, id: 4)
```

Use code with caution.

2. **Update the task to completed status:**

```bash
curl -i -X PUT -H "Content-Type: application/json" -d '{"done":true}' http://localhost
```

Use code with caution.

Expected Outcome: Status **200 OK** returning the task with `"done": true`.

3. **Restart your server process** (`Ctrl + C` followed by restart command).
4. **Delete the task completely:**

```bash
curl -i -X DELETE http://localhost:3000/tasks/4
```

Use code with caution.

Expected Outcome: Status **204 No Content** with an empty response body.

5. **Verify deletion with an unknown lookup request:**

```bash
curl -i http://localhost:3000/tasks/4
```

Use code with caution.

Expected Outcome: Status **404 Not Found**.

💾 Git Commit

Now that you have built a complete, functional, database-backed CRUD API, save your work with a final commit:

bash

```
git add server.js main.py
git commit -m "Stage 3: update and delete with SQL"
git push
```